

Memory-Mapped I/O

1 Controller Registers

Main Idea

Each device controller has registers that the CPU uses to communicate with the device.

The operating system writes to controller registers to command the device. It reads from them to check device status.

Register use	Meaning
Command	Tell the device to read, write, start, stop, or perform another action.
Status	Check whether the device is ready, busy, finished, or in error.
Data buffer	Hold data being transferred between the device and memory.

Example: a display may use video RAM as a data buffer. Programs or the operating system write pixel data into this buffer, and the display controller uses it to update the screen.

2 Separate I/O Port Space

Definition

In **separate I/O port space**, device registers are addressed by I/O port numbers instead of normal memory addresses.

An I/O port number is commonly an 8-bit or 16-bit integer. The set of all I/O ports is called the **I/O port space**. It is protected so ordinary user programs cannot access it directly.

```
IN  REG,PORT    // read I/O port PORT into CPU register REG
OUT PORT,REG     // write CPU register REG into I/O port PORT
```

In this design, memory addresses and I/O port addresses are separate.

Instruction	Meaning
IN R0,4	Read I/O port 4 into register <i>R0</i> .
MOV R0,4	Read memory word 4 into register <i>R0</i> .

Key Point

The two 4's refer to different address spaces: one is an I/O port address, and the other is a memory address.

3 Memory-Mapped I/O

Definition

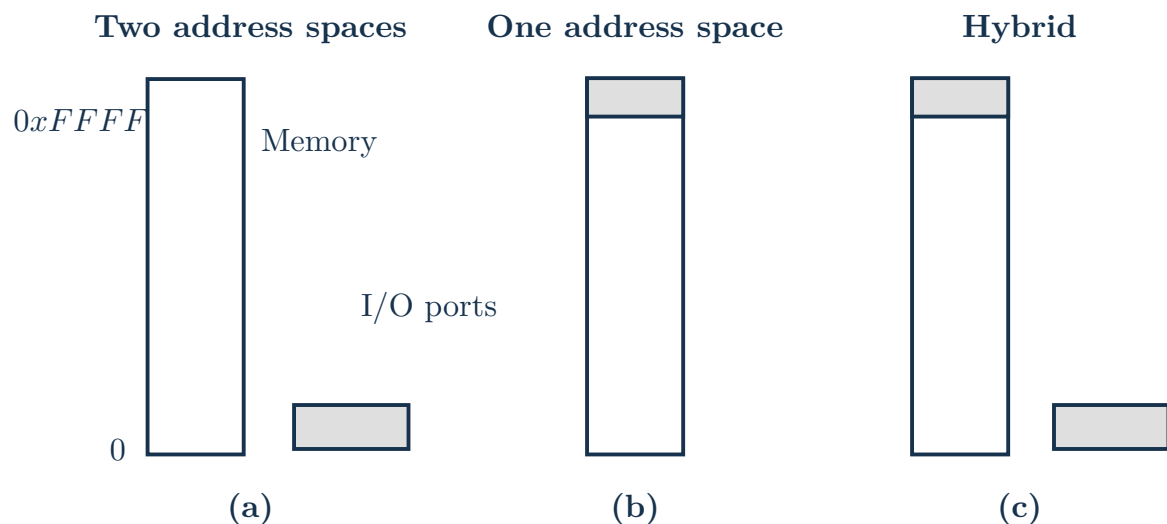
In **memory-mapped I/O**, device control registers are assigned normal memory addresses.

The assigned addresses are not backed by ordinary RAM. Instead, when the CPU reads or writes those addresses, the device controller responds. These addresses are usually placed near the top of the address space.

Simple View

With memory-mapped I/O, device registers look like memory variables to the CPU and to system software.

4 Addressing Schemes



Scheme	Meaning
Separate I/O	Memory and I/O ports use different address spaces.
Memory-mapped I/O	Device registers occupy selected addresses inside the memory address space.
Hybrid	Data buffers are memory-mapped, but control registers use separate I/O ports.

x86 Example

x86 systems use a hybrid approach: some device buffers are memory-mapped, and separate I/O ports are also available.

5 How the CPU Accesses I/O

For a read operation, the CPU places an address on the bus and asserts a read control signal.

Design	Who responds?
Separate spaces	A control signal tells whether the address is for memory or I/O. Memory responds to memory requests; the I/O device responds to I/O requests.
One address space	Memory modules and I/O devices compare the address with their assigned ranges. The matching component responds.

No Address Conflict

In memory-mapped I/O, no address should be assigned to both RAM and an I/O device. Otherwise, two components could respond to the same CPU request.

6 Advantages of Memory-Mapped I/O

Advantage	Explanation
Can use C/C++	Device registers can be accessed like memory variables. Special assembly instructions are not required.
Simpler protection	The OS can keep device-register pages out of user virtual address spaces.
Fine-grained access	The OS can map only selected device pages into a process or driver address space.
Normal instructions work	Any instruction that reads or writes memory can also read or write device registers.

```

LOOP:  TEST PORT4      // check if device register is 0
        BEQ  READY     // if 0, device is ready
        BRANCH LOOP    // otherwise keep testing
READY:

```

Instruction Benefit

With memory-mapped I/O, a register can be tested directly. With separate I/O ports, the value must first be read into the CPU and then tested.

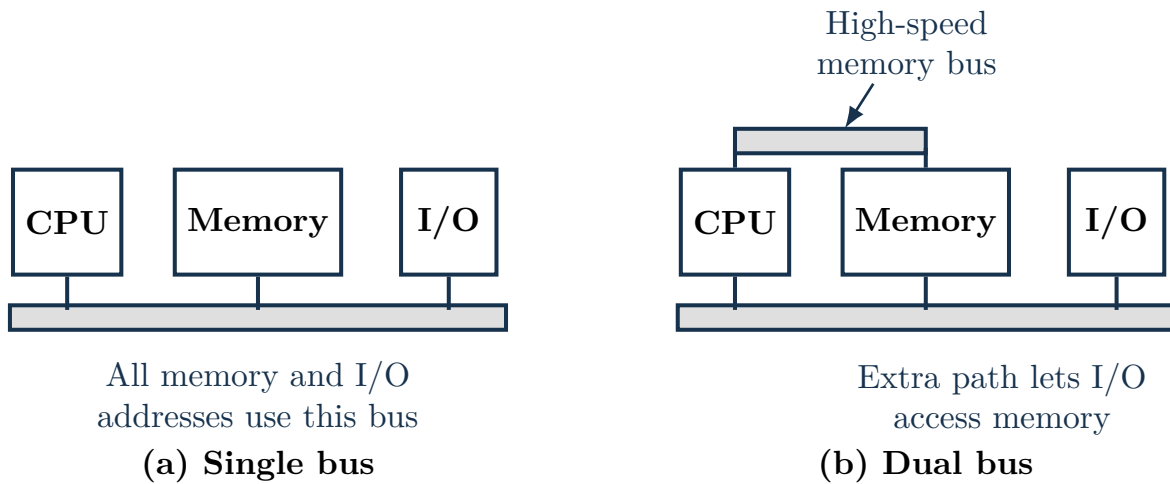
7 Disadvantages of Memory-Mapped I/O

Problem	Explanation
Caching danger	A cached device register may hide a real device-status change.
Hardware complexity	The hardware must disable caching for I/O address ranges, often per page.
Bus visibility	With one address space, I/O devices may need to inspect memory references.
Multiple buses	Modern systems often have separate high-speed memory buses, so I/O devices may not see memory addresses directly.

Caching Problem

Device control registers must not be cached like normal memory. If a status register is cached, the CPU may keep reading the old value and never notice that the device became ready.

8 Bus Design Issues



Possible Solutions

On systems with separate memory and I/O buses, memory-mapped I/O needs extra support.

- Try memory first; if memory does not respond, try the I/O buses.
- Add a snooping device that forwards relevant memory-bus addresses to I/O devices.
- Let the memory controller filter address ranges and forward nonmemory addresses to devices.

Final Point

Memory-mapped I/O simplifies software, but it adds hardware and caching trade-offs.